

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE CODE: MLA0304

COURSE NAME: Reinforcement learning

COURSE OUTCOME COVERED

CO4: Evaluate model-based reinforcement learning approaches for prediction, planning, and policy optimization. (BL3,BL4,BL5)

Assessment Tool Weightage: 40%

Dr . DUMPALA PRASANTH

QUESTIONS: 25

Total Marks: 50

Annexure A

Design and Develop Model

DURATION: 4 HRS (2 Sessions)

Model-Based RL Approach

Draw a flowchart illustrating the complete workflow of a Model-Based Reinforcement Learning agent from environment interaction to policy optimization.

Draw a concept map showing the relationship between the Environment, Model, Planning Module, Policy, and Reward in Model-Based Reinforcement Learning.

Design a flowchart explaining how an RL agent learns an environment model before selecting an optimal action.

Draw the architecture of a Model-Based RL system showing model learning, planning, and execution components.

Prepare a concept map comparing Model-Based Reinforcement Learning and Model-Free Reinforcement Learning.

Analytic Gradient Computation

Draw a flowchart explaining the steps involved in analytic gradient computation for policy optimization.

Design a concept map showing how gradients are computed and propagated during policy learning.

Draw a block diagram illustrating gradient computation, parameter updates, and reward optimization in Reinforcement Learning.

Prepare a flowchart explaining gradient-based optimization using backpropagation in Reinforcement Learning

(5 Marks)

Draw a concept map showing the relationship among policy parameters, gradients, optimization algorithms, and rewards.

(5 Marks)

Sampling-Based Planning

Draw a flowchart illustrating the sampling-based planning process in Model-Based Reinforcement Learning.

(5 Marks)

Prepare a concept map showing how sampled trajectories are generated and evaluated for planning.

(5 Marks)

Draw a block diagram showing the interaction between the environment model, planner, sampled actions, and optimal policy.

(5 Marks)

Design a flowchart explaining Monte Carlo sampling for planning in Reinforcement Learning.

(5 Marks)

Draw a concept map comparing exhaustive search planning and sampling-based planning techniques.

(5 Marks)

Model-Based Data Generation

Draw a flowchart illustrating synthetic data generation using a learned environment model.

(5 Marks)

Prepare a concept map showing the relationship among environment model, generated experiences, replay buffer, and policy learning.

(5 Marks)

Draw a block diagram explaining model-generated experience replay in Reinforcement Learning

(5 Marks)

Design a flowchart showing how simulated experiences improve sample efficiency during training.

(5 Marks)

Draw a concept map illustrating the advantages and limitations of model-generated training data.

(5 Marks)

Value-Equivalence Prediction and Policy Optimization

Draw a flowchart explaining value-equivalence prediction for estimating future rewards

(5 Marks)

Prepare a concept map showing the relationship among value functions, reward prediction, policy evaluation, and policy improvement.

(5 Marks)

Draw a block diagram illustrating the complete Model-Based Policy Optimization (MBPO) framework.

(5 Marks)

Design a flowchart explaining iterative policy optimization using learned environment dynamics.

(5 Marks)

Draw a comprehensive concept map integrating Model-Based RL, Analytic Gradient Computation, Sampling-Based Planning, Model-Based Data Generation, Value-Equivalence Prediction, and Model-Based Policy Optimization into a unified Reinforcement Learning framework.

Code of Conduct:

I (V Sunil Kumar / 192425292) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

V Sunil Kumar

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

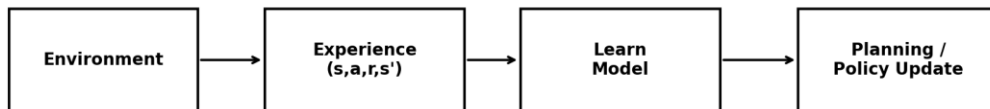
MODEL-BASED RL APPROACH

1. Draw a flowchart illustrating the complete workflow of a Model-Based Reinforcement Learning agent from environment interaction to policy optimization.

Answer / Design Explanation

A Model-Based Reinforcement Learning (MBRL) agent first interacts with the environment to collect transitions such as state, action, reward, and next state. These transitions are used to learn an approximate environment model. The model predicts future states and rewards for candidate actions. A planning module then uses these predictions to evaluate possible action sequences. The selected action is executed in the real environment, generating new experience. The process is repeated, and the policy and model are continuously refined. This approach can improve sample efficiency because planning can reuse learned dynamics rather than relying only on expensive real interactions. Policy optimization may use value estimates, trajectory returns, gradients, or other planning objectives. The complete loop is: interact → collect data → learn dynamics → simulate/planning → optimize policy → execute → collect new data.

Model-Based RL: Complete Workflow



Execute policy → collect new experience → update model and policy

Figure 1.1 — Primary diagram for Question 1.

Component	Role / Process	Expected Outcome
Environment	Provides states, actions and rewards	Real experience
Model / Planner	Predicts futures and evaluates actions	Candidate decisions
Policy	Selects or improves actions	Higher expected return

Table 1.1 — Key components and expected outcomes for the proposed design.

Policy Optimization Feedback Loop

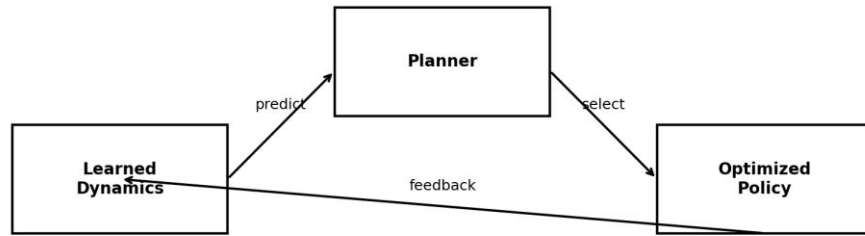


Figure 1.2 — Supporting relationship / process diagram for Question 1.

2. Draw a concept map showing the relationship between the Environment, Model, Planning Module, Policy, and Reward in Model-Based Reinforcement Learning.

Answer / Design Explanation

The main MBRL elements are connected as a feedback system. The Environment produces real transitions and rewards. The Model learns the environment dynamics from those transitions and predicts what may happen after actions. The Planning Module uses the model to simulate candidate trajectories and estimate their outcomes. The Policy converts the current state into an action and is improved using information from planning. The Reward function measures the quality of outcomes and provides the objective for policy improvement. Thus, Environment $\rightarrow$ Model $\rightarrow$ Planning $\rightarrow$ Policy $\rightarrow$ Environment forms the main loop, while Reward influences both planning evaluation and policy optimization. A model may predict transition probabilities and expected rewards, while the planner searches for actions that maximize expected return.

Concept Map: Core MBRL Elements

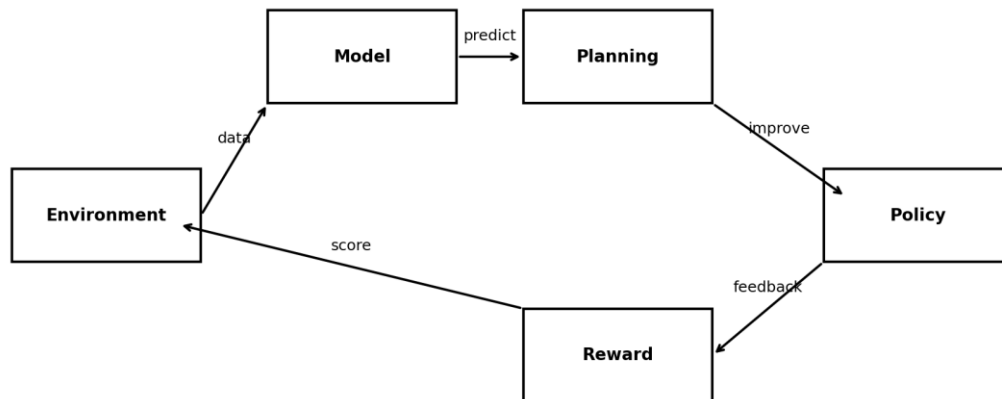


Figure 2.1 — Primary diagram for Question 2.

Component	Role / Process	Expected Outcome
Environment	Provides states, actions and rewards	Real experience
Model / Planner	Predicts futures and evaluates actions	Candidate decisions
Policy	Selects or improves actions	Higher expected return

Table 2.1 — Key components and expected outcomes for the proposed design.

MBRL Relationship Map

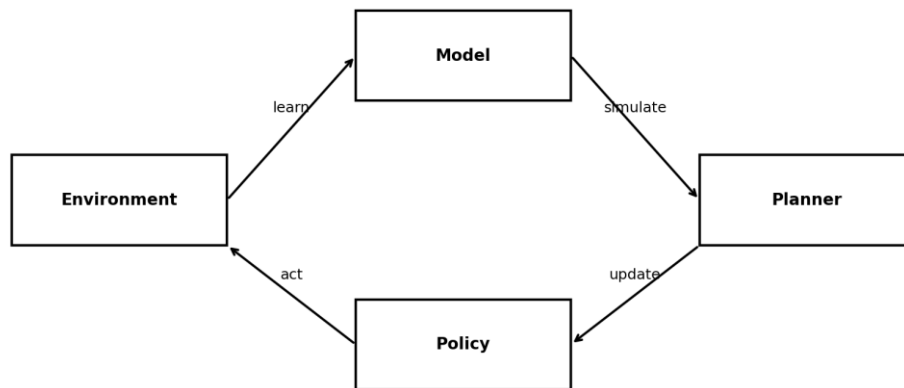


Figure 2.2 — Supporting relationship / process diagram for Question 2.

3. Design a flowchart explaining how an RL agent learns an environment model before selecting an optimal action.

Answer / Design Explanation

The agent first observes the current state and takes exploratory actions. Each transition is stored as experience. After enough transitions are available, a dynamics model is fitted to predict next states and rewards. The learned model is checked against observed transitions to reduce prediction error. Before selecting the next action, the agent can simulate several candidate actions through the model. The predicted trajectories are evaluated using rewards or value estimates. The action with the highest expected return is selected, subject to safety and action constraints. The chosen action is then executed in the real environment. New real data are added to the dataset and the model is updated. This creates an iterative learning-and-planning cycle.

Learning the Environment Model Before Acting

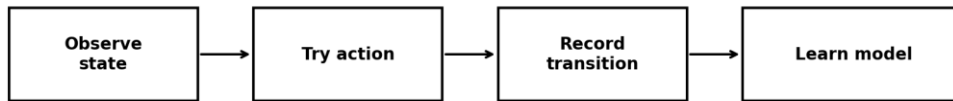


Figure 3.1 — Primary diagram for Question 3.

Component	Role / Process	Expected Outcome
Environment	Provides states, actions and rewards	Real experience
Model / Planner	Predicts futures and evaluates actions	Candidate decisions
Policy	Selects or improves actions	Higher expected return

Table 3.1 — Key components and expected outcomes for the proposed design.

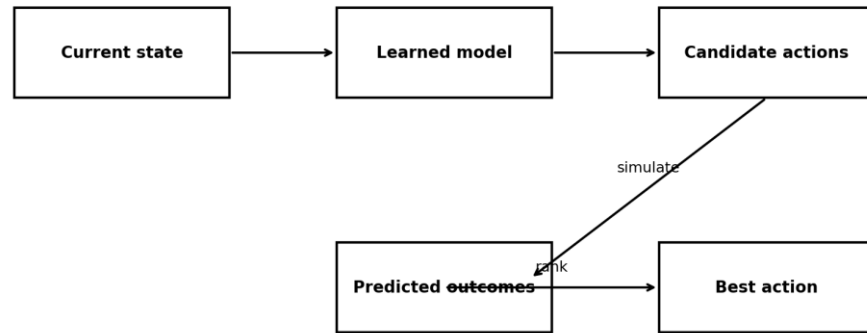
Model-Guided Action Selection

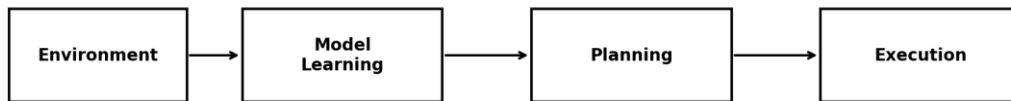
Figure 3.2 — Supporting relationship / process diagram for Question 3.

4. Draw the architecture of a Model-Based RL system showing model learning, planning, and execution components.

Answer / Design Explanation

A typical Model-Based RL architecture has four major components: environment, model learning, planning, and execution. The environment supplies real observations and rewards. The model-learning component estimates transition dynamics and reward behavior from experience. The planning component uses the learned model to generate possible futures and choose promising actions or policy updates. The execution component sends the selected action back to the real environment. The system then receives new experience and repeats the loop. This architecture separates learning of dynamics from decision-making and allows simulated experience to support policy optimization.

Model-Based RL Architecture



Real experience → learned dynamics → simulated planning → action

Figure 4.1 — Primary diagram for Question 4.

Component	Role / Process	Expected Outcome
Environment	Provides states, actions and rewards	Real experience
Model / Planner	Predicts futures and evaluates actions	Candidate decisions
Policy	Selects or improves actions	Higher expected return

Table 4.1 — Key components and expected outcomes for the proposed design.

Three-Layer View

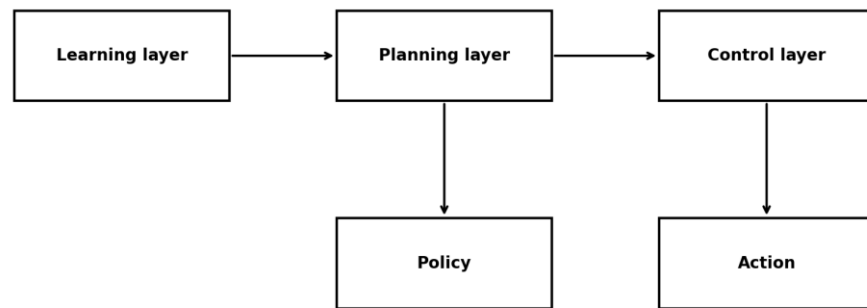


Figure 4.2 — Supporting relationship / process diagram for Question 4.

5. Prepare a concept map comparing Model-Based Reinforcement Learning and Model-Free Reinforcement Learning.

Answer / Design Explanation

Model-Based RL explicitly learns or uses an environment model and performs planning with it. Model-Free RL does not build an explicit dynamics model; it learns a policy or value function directly from experience. MBRL can be more sample efficient because one observed transition can support many model-based simulations, but inaccurate models can produce planning errors. Model-Free methods are often simpler to deploy and can work well with large amounts of interaction data, but they may require substantially more samples. MBRL is attractive when real interactions are expensive and planning is useful. Model-Free RL is attractive when experience is abundant and direct policy learning is sufficient. Hybrid methods can combine both approaches.

Model-Based vs Model-Free RL

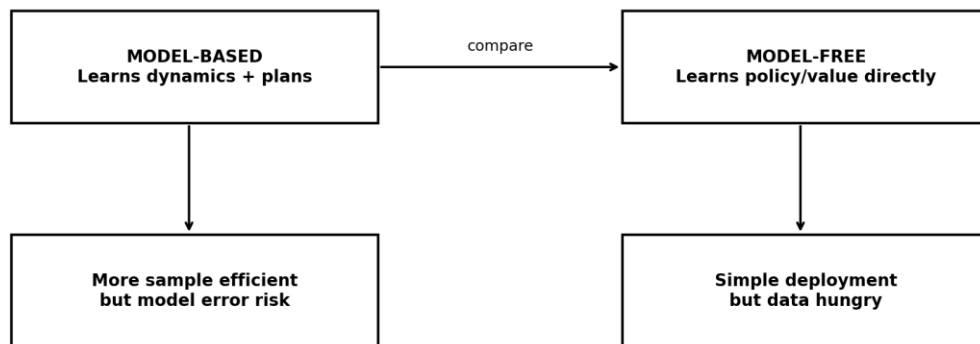


Figure 5.1 — Primary diagram for Question 5.

Component	Role / Process	Expected Outcome
Environment	Provides states, actions and rewards	Real experience
Model / Planner	Predicts futures and evaluates actions	Candidate decisions
Policy	Selects or improves actions	Higher expected return

Table 5.1 — Key components and expected outcomes for the proposed design.

Comparison Decision Guide

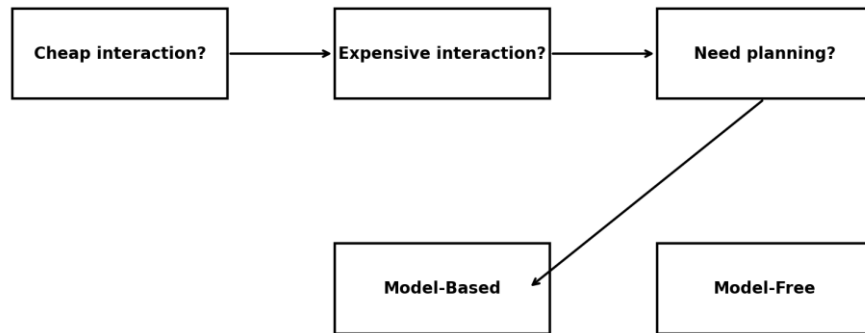


Figure 5.2 — Supporting relationship / process diagram for Question 5.

ANALYTIC GRADIENT COMPUTATION

6. Draw a flowchart explaining the steps involved in analytic gradient computation for policy optimization.

Answer / Design Explanation

Analytic gradient computation begins by defining a differentiable policy with parameters θ . The agent interacts with the environment and collects states, actions, and rewards. The learning objective, such as expected return $J(\theta)$, is computed from the collected trajectory. The algorithm derives the gradient of the objective with respect to θ . For a policy-gradient method, a common form is $\nabla J(\theta) = E[\nabla \theta \log \pi_{\theta}(a|s) A(s,a)]$, where A is an advantage estimate. Automatic differentiation can calculate the required derivatives through the computational graph. The optimizer then updates θ in the direction that improves the objective. The process repeats with new experience.

Analytic Gradient Computation Flow



Figure 6.1 — Primary diagram for Question 6.

Component	Role / Process	Expected Outcome
Parameters θ	Represent the policy	Current behavior
Gradient	Measures objective direction	Update signal
Optimizer	Changes θ using the gradient	Improved policy

Table 6.1 — Key components and expected outcomes for the proposed design.

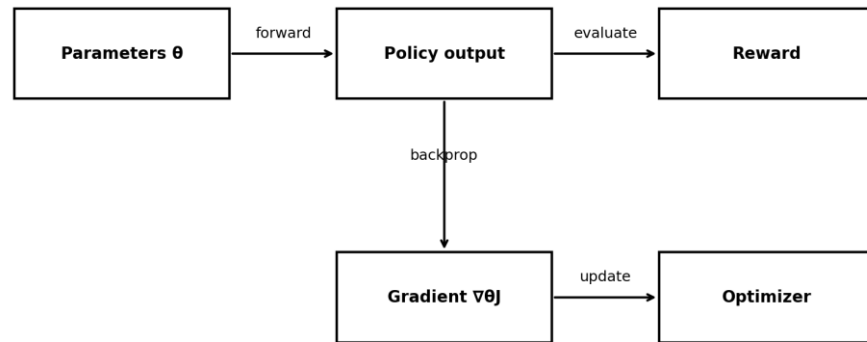
Gradient Learning Concept

Figure 6.2 — Supporting relationship / process diagram for Question 6.

7. Design a concept map showing how gradients are computed and propagated during policy learning.

Answer / Design Explanation

During policy learning, the policy parameters θ produce actions from observed states. The resulting rewards are used to calculate a learning objective. Backpropagation applies the chain rule through the policy network to calculate how the objective changes with respect to each parameter. The resulting gradient vector identifies the direction of improvement. An optimizer such as SGD or Adam uses this gradient to update the parameters. In actor–critic learning, a critic supplies value or advantage estimates that improve the quality of the policy gradient. Gradient clipping, normalization, and stable learning rates can reduce unstable updates.

Gradient Propagation in Policy Learning



Figure 7.1 — Primary diagram for Question 7.

Component	Role / Process	Expected Outcome
Parameters θ	Represent the policy	Current behavior
Gradient	Measures objective direction	Update signal
Optimizer	Changes θ using the gradient	Improved policy

Table 7.1 — Key components and expected outcomes for the proposed design.

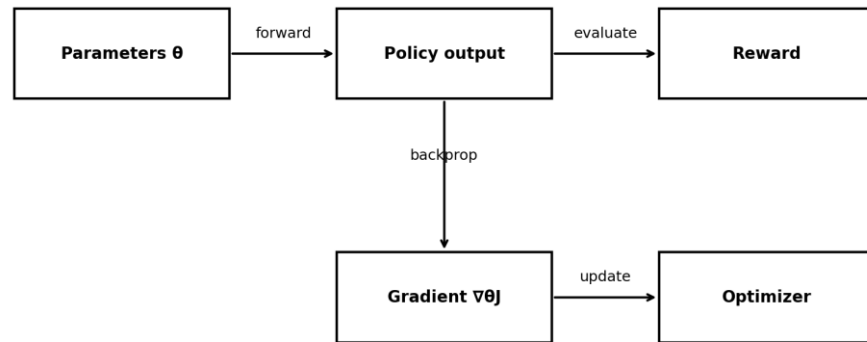
Gradient Learning Concept

Figure 7.2 — Supporting relationship / process diagram for Question 7.

8. Draw a block diagram illustrating gradient computation, parameter updates, and reward optimization in Reinforcement Learning.

Answer / Design Explanation

The RL gradient block diagram begins with state input to the policy network. The network produces an action distribution or action value. Interaction with the environment produces rewards and new states. A return or loss is calculated from these outcomes. Analytic or automatic differentiation calculates the gradient of the objective with respect to policy parameters. The optimizer updates the parameters, creating a new policy. Repeated interaction gradually increases expected reward. The block structure therefore connects observation, action selection, reward evaluation, gradient computation, parameter update, and improved policy.

Gradient → Update → Reward Optimization



Figure 8.1 — Primary diagram for Question 8.

Component	Role / Process	Expected Outcome
Parameters θ	Represent the policy	Current behavior
Gradient	Measures objective direction	Update signal
Optimizer	Changes θ using the gradient	Improved policy

Table 8.1 — Key components and expected outcomes for the proposed design.

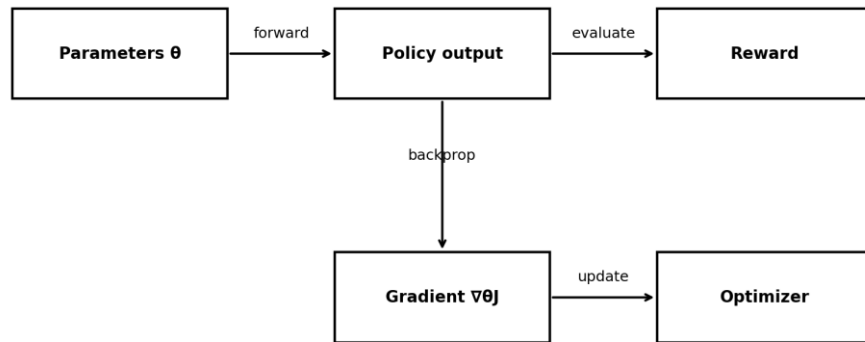
Gradient Learning Concept

Figure 8.2 — Supporting relationship / process diagram for Question 8.

9. Prepare a flowchart explaining gradient-based optimization using backpropagation in Reinforcement Learning.

Answer / Design Explanation

Backpropagation in RL follows the same chain-rule principle used in neural-network training, but the learning signal comes from rewards or value estimates. The policy receives an observation and produces an action. After interaction, a return or loss is calculated. The loss is represented as a computational graph involving policy outputs and parameters. Backpropagation calculates partial derivatives from the output layer back through hidden layers to the policy parameters. The optimizer applies these gradients to update the network. In policy-gradient methods, the gradient is weighted by the quality of the sampled action. Repeating this process improves the policy while regularization and clipping can improve stability.

Backpropagation in RL



Figure 9.1 — Primary diagram for Question 9.

Component	Role / Process	Expected Outcome
Parameters θ	Represent the policy	Current behavior
Gradient	Measures objective direction	Update signal
Optimizer	Changes θ using the gradient	Improved policy

Table 9.1 — Key components and expected outcomes for the proposed design.

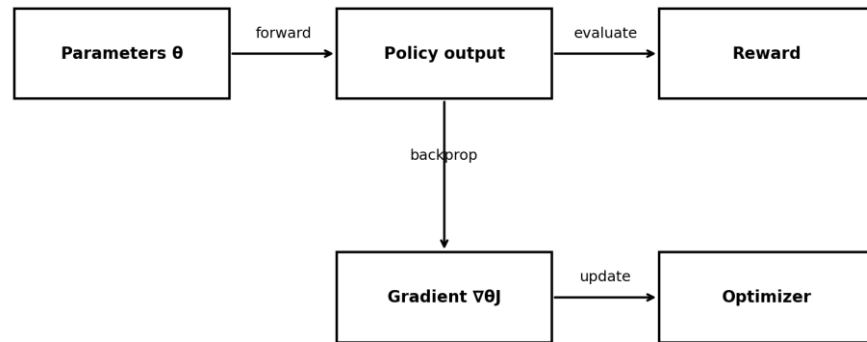
Gradient Learning Concept

Figure 9.2 — Supporting relationship / process diagram for Question 9.

10. Draw a concept map showing the relationship among policy parameters, gradients, optimization algorithms, and rewards.

Answer / Design Explanation

Policy parameters θ define the behavior of the RL policy. The policy maps states to actions or action probabilities. Rewards from the environment are used to estimate the quality of that behavior. An objective function is constructed from returns, advantages, or another learning signal. Differentiation produces $\nabla \theta J$, showing how each parameter should change to improve expected performance. Optimization algorithms such as SGD, Adam, PPO updates, or other constrained methods then modify θ . The new policy interacts with the environment again, producing new rewards. This forms a continuous parameter $\rightarrow$ action $\rightarrow$ reward $\rightarrow$ gradient $\rightarrow$ optimization cycle.

Policy Parameters and Gradients



Figure 10.1 — Primary diagram for Question 10.

Component	Role / Process	Expected Outcome
Parameters θ	Represent the policy	Current behavior
Gradient	Measures objective direction	Update signal
Optimizer	Changes θ using the gradient	Improved policy

Table 10.1 — Key components and expected outcomes for the proposed design.

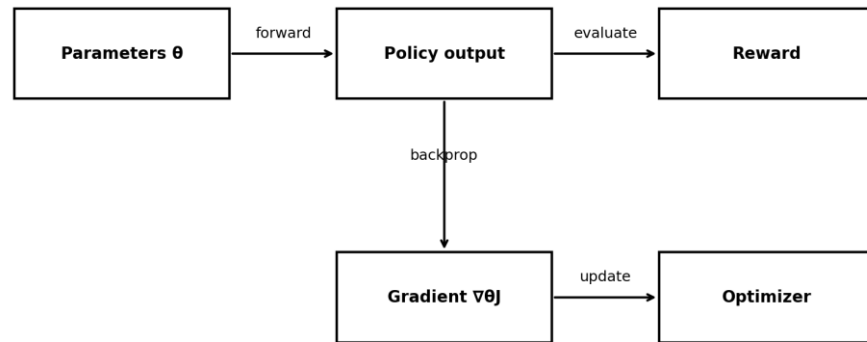
Gradient Learning Concept

Figure 10.2 — Supporting relationship / process diagram for Question 10.

SAMPLING-BASED PLANNING

11. Draw a flowchart illustrating the sampling-based planning process in Model-Based Reinforcement Learning.

Answer / Design Explanation

Sampling-based planning begins with a learned environment model and the current state. Instead of exhaustively enumerating every possible action sequence, the planner samples a manageable number of candidate actions or trajectories. Each sample is rolled forward through the model to produce predicted states and rewards. The cumulative return of each trajectory is calculated. The planner compares these sampled returns and selects the best action or uses the samples to improve a policy. New real experience can update the model, and planning is repeated. Sampling reduces computational cost when the full search space is too large.

Sampling-Based Planning: 11

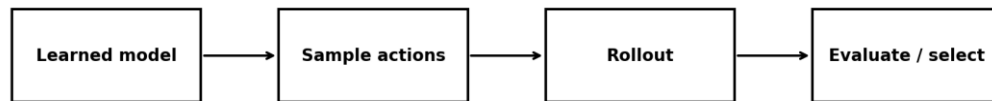


Figure 11.1 — Primary diagram for Question 11.

Component	Role / Process	Expected Outcome
Sampling	Creates candidate trajectories	Search coverage
Rollout	Predicts future states/rewards	Trajectory return
Selection	Ranks sampled outcomes	Best action/policy

Table 11.1 — Key components and expected outcomes for the proposed design.

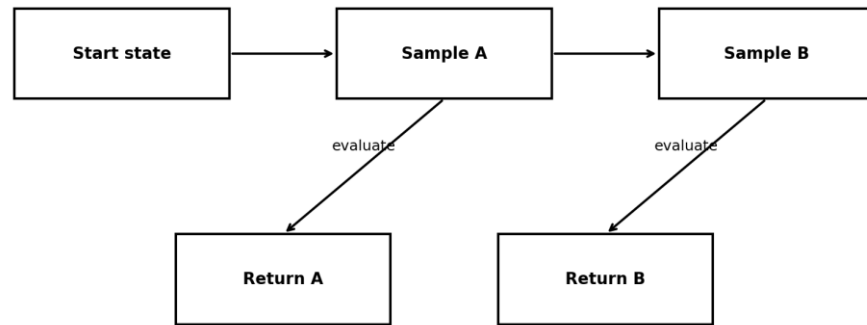
Planning Search Space

Figure 11.2 — Supporting relationship / process diagram for Question 11.

12. Prepare a concept map showing how sampled trajectories are generated and evaluated for planning.

Answer / Design Explanation

Sampled trajectories are generated by choosing candidate actions according to a sampling strategy. Each action is passed through the learned model to predict the next state and reward. This process continues for a selected planning horizon. The resulting trajectory contains states, actions, and predicted rewards. A return is computed for each trajectory, possibly using a discount factor. The planner evaluates the samples and identifies high-value actions. Repeated sampling increases the chance of discovering strong decisions without requiring exhaustive search.

Sampling-Based Planning: 12

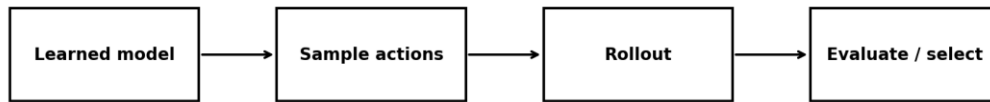


Figure 12.1 — Primary diagram for Question 12.

Component	Role / Process	Expected Outcome
Sampling	Creates candidate trajectories	Search coverage
Rollout	Predicts future states/rewards	Trajectory return
Selection	Ranks sampled outcomes	Best action/policy

Table 12.1 — Key components and expected outcomes for the proposed design.

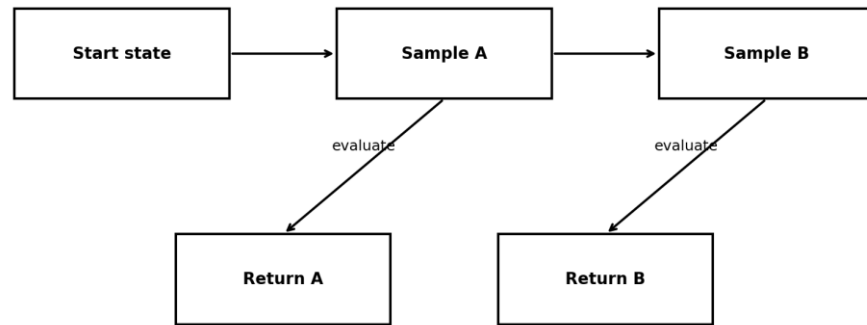
Planning Search Space

Figure 12.2 — Supporting relationship / process diagram for Question 12.

13. Draw a block diagram showing the interaction between the environment model, planner, sampled actions, and optimal policy.

Answer / Design Explanation

The environment model predicts transitions and rewards. The planner queries this model with sampled candidate actions. Each sampled action generates a predicted trajectory. The planner evaluates trajectories using cumulative reward or a value function. The best action, or a distribution over good actions, is passed to the policy optimization component. The selected action is then executed in the real environment. New observations and rewards are used to update both the model and policy. Thus the model and planner provide a bridge between real experience and policy improvement.

Sampling-Based Planning: 13

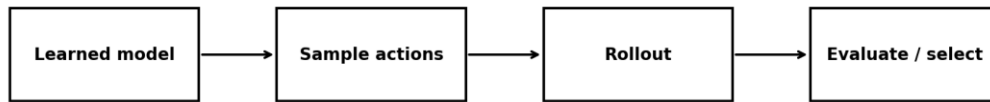


Figure 13.1 — Primary diagram for Question 13.

Component	Role / Process	Expected Outcome
Sampling	Creates candidate trajectories	Search coverage
Rollout	Predicts future states/rewards	Trajectory return
Selection	Ranks sampled outcomes	Best action/policy

Table 13.1 — Key components and expected outcomes for the proposed design.

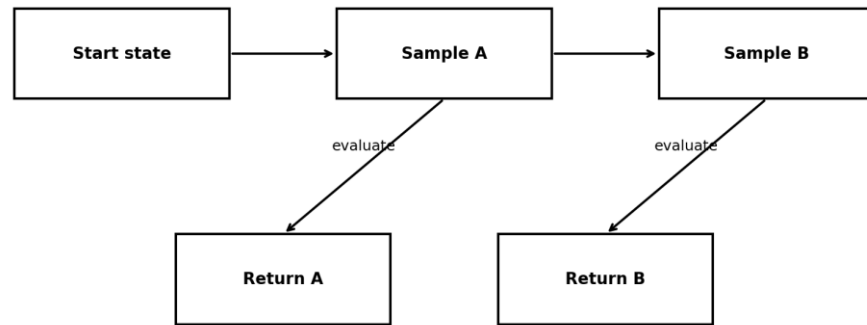
Planning Search Space

Figure 13.2 — Supporting relationship / process diagram for Question 13.

14. Design a flowchart explaining Monte Carlo sampling for planning in Reinforcement Learning.

Answer / Design Explanation

Monte Carlo planning generates complete or partial sampled trajectories from a current state. A policy or sampling distribution chooses an action, the learned model predicts the next state and reward, and the process continues until the planning horizon or terminal state. The return of each sampled trajectory is calculated. Many samples are averaged or compared to estimate the value of actions. The planner then selects the action with the highest estimated value or improves the policy toward high-return samples. Increasing the number of samples can improve the estimate but also increases computation.

Sampling-Based Planning: 14

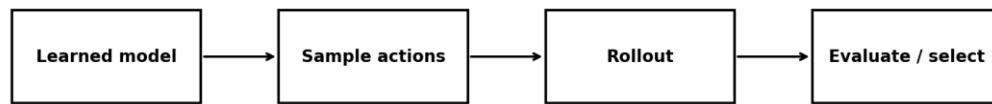


Figure 14.1 — Primary diagram for Question 14.

Component	Role / Process	Expected Outcome
Sampling	Creates candidate trajectories	Search coverage
Rollout	Predicts future states/rewards	Trajectory return
Selection	Ranks sampled outcomes	Best action/policy

Table 14.1 — Key components and expected outcomes for the proposed design.

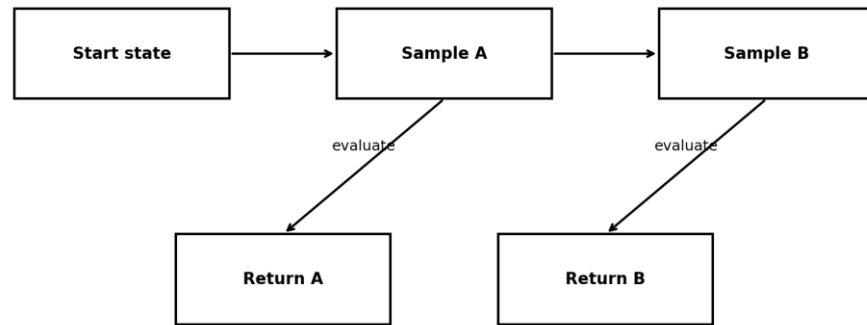
Planning Search Space

Figure 14.2 — Supporting relationship / process diagram for Question 14.

15. Draw a concept map comparing exhaustive search planning and sampling-based planning techniques.

Answer / Design Explanation

Exhaustive search systematically considers all or a very large fraction of possible action sequences. It can provide strong planning results in small search spaces but becomes computationally expensive as the horizon and action space grow. Sampling-based planning instead explores a selected subset of possible trajectories. It can handle larger search spaces and provides a controllable trade-off between computation and planning quality. However, sampling can miss rare but excellent trajectories. Techniques such as importance sampling, guided sampling, model-predictive control, and learned proposal distributions can improve sample efficiency.

Sampling-Based Planning: 15

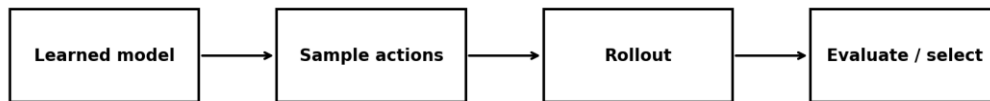


Figure 15.1 — Primary diagram for Question 15.

Component	Role / Process	Expected Outcome
Sampling	Creates candidate trajectories	Search coverage
Rollout	Predicts future states/rewards	Trajectory return
Selection	Ranks sampled outcomes	Best action/policy

Table 15.1 — Key components and expected outcomes for the proposed design.

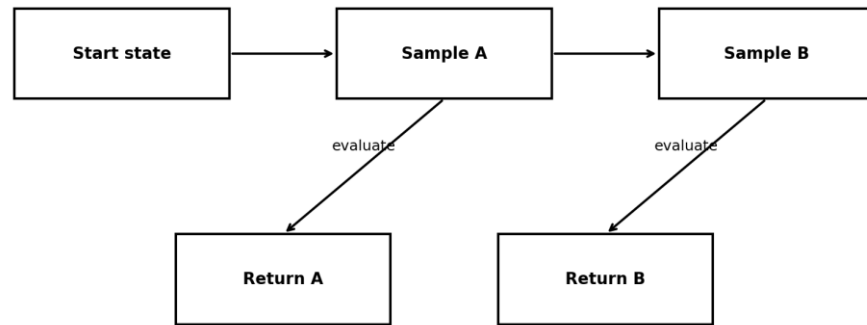
Planning Search Space

Figure 15.2 — Supporting relationship / process diagram for Question 15.

MODEL-BASED DATA GENERATION

16. Draw a flowchart illustrating synthetic data generation using a learned environment model.

Answer / Design Explanation

Synthetic data generation begins with real environment transitions. A dynamics model is trained to predict next states and rewards. Once its predictions are sufficiently accurate, the model is initialized with real states and rolled forward using sampled or policy-generated actions. The resulting synthetic transitions form simulated experience. These transitions can be added to a replay buffer and used for policy or value learning. Periodically, real environment data are collected and the model is retrained or corrected. This creates a hybrid data pipeline where real experience anchors the model and synthetic experience expands training data.

Model-Based Data Generation: 16

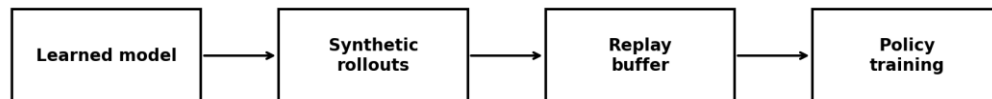


Figure 16.1 — Primary diagram for Question 16.

Component	Role / Process	Expected Outcome
Learned model	Generates predicted transitions	Synthetic experience
Replay buffer	Stores real + model data	Reusable samples
Policy learner	Trains from sampled batches	Better sample efficiency

Table 16.1 — Key components and expected outcomes for the proposed design.

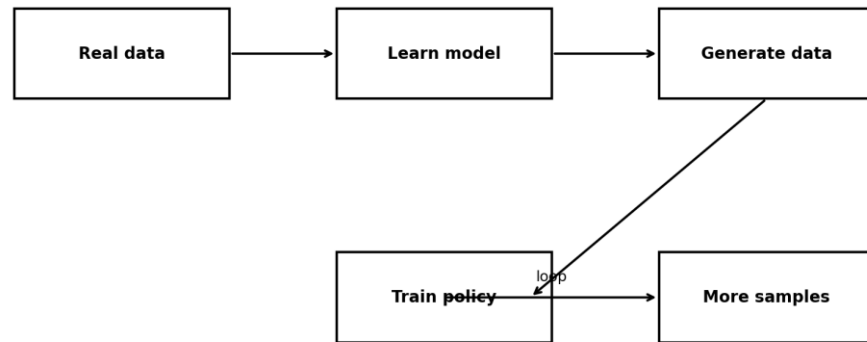
Synthetic Experience Loop

Figure 16.2 — Supporting relationship / process diagram for Question 16.

17. Prepare a concept map showing the relationship among environment model, generated experiences, replay buffer, and policy learning.

Answer / Design Explanation

The environment provides real transitions that are used to train the learned model. The model generates additional experiences by simulating policy actions from real or previously observed states. Generated experiences are stored in a replay buffer together with real experiences, or in a separate model-generated buffer. The policy learner samples batches from these experiences to update the policy or value function. The improved policy creates new actions, which can be evaluated in the model and eventually in the real environment. This architecture increases sample efficiency but requires monitoring for model bias.

Model-Based Data Generation: 17

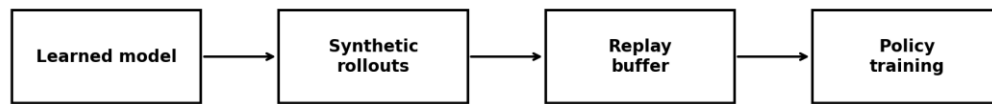


Figure 17.1 — Primary diagram for Question 17.

Component	Role / Process	Expected Outcome
Learned model	Generates predicted transitions	Synthetic experience
Replay buffer	Stores real + model data	Reusable samples
Policy learner	Trains from sampled batches	Better sample efficiency

Table 17.1 — Key components and expected outcomes for the proposed design.

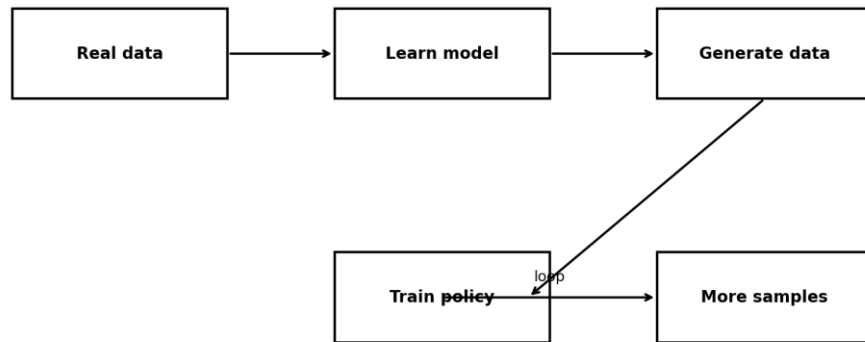
Synthetic Experience Loop

Figure 17.2 — Supporting relationship / process diagram for Question 17.

18. Draw a block diagram explaining model-generated experience replay in Reinforcement Learning.

Answer / Design Explanation

Model-generated experience replay contains a learned model, a state sampler, a rollout mechanism, a replay buffer, and a policy learner. The model predicts transitions from selected starting states. Short synthetic rollouts are generated and stored. The learner samples mini-batches from the buffer and updates the policy or value network. Real experience can be mixed with synthetic experience to reduce model bias. Short model rollouts are often preferred because prediction errors can compound over long horizons.

Model-Based Data Generation: 18

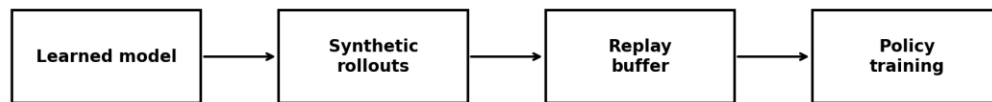


Figure 18.1 — Primary diagram for Question 18.

Component	Role / Process	Expected Outcome
Learned model	Generates predicted transitions	Synthetic experience
Replay buffer	Stores real + model data	Reusable samples
Policy learner	Trains from sampled batches	Better sample efficiency

Table 18.1 — Key components and expected outcomes for the proposed design.

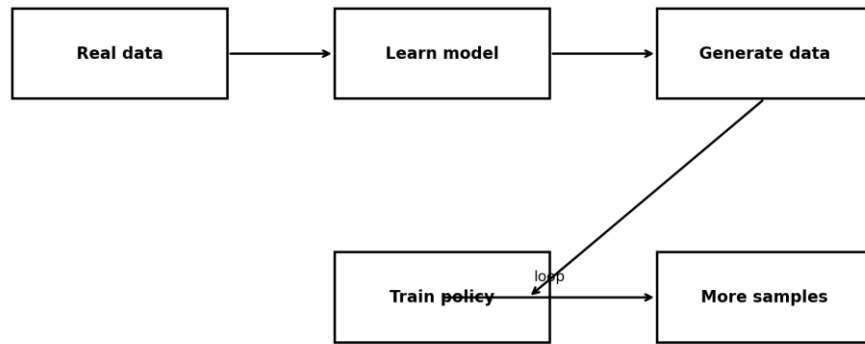
Synthetic Experience Loop

Figure 18.2 — Supporting relationship / process diagram for Question 18.

19. Design a flowchart showing how simulated experiences improve sample efficiency during training.

Answer / Design Explanation

Simulated experiences improve sample efficiency by allowing the agent to learn from many predicted transitions for each expensive real interaction. The agent first collects real data and learns an environment model. The model is then used to generate short trajectories. These trajectories are inserted into training data and used to update the policy. After policy improvement, the agent collects additional real data, checks model accuracy, and repeats the process. If model error becomes high, the system reduces reliance on synthetic data and collects more real transitions.

Model-Based Data Generation: 19

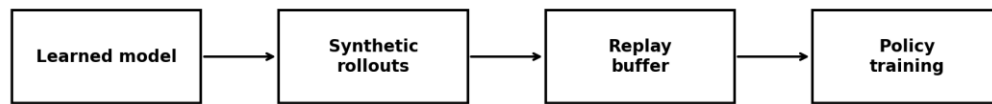


Figure 19.1 — Primary diagram for Question 19.

Component	Role / Process	Expected Outcome
Learned model	Generates predicted transitions	Synthetic experience
Replay buffer	Stores real + model data	Reusable samples
Policy learner	Trains from sampled batches	Better sample efficiency

Table 19.1 — Key components and expected outcomes for the proposed design.

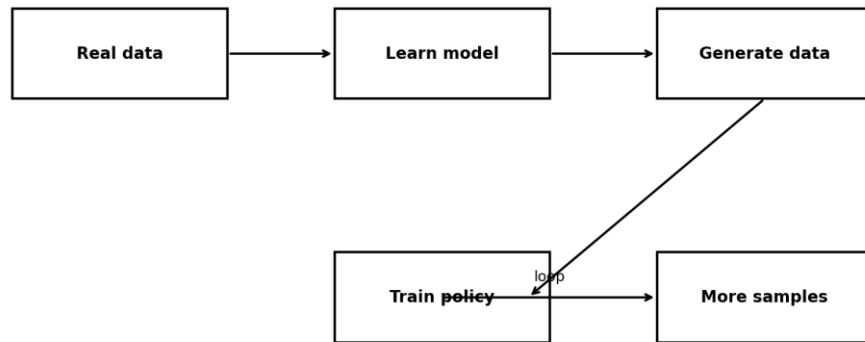
Synthetic Experience Loop

Figure 19.2 — Supporting relationship / process diagram for Question 19.

20. Draw a concept map illustrating the advantages and limitations of model-generated training data.

Answer / Design Explanation

Advantages of model-generated data include greater sample efficiency, lower real-world interaction cost, faster experimentation, and the ability to generate rare scenarios. It can also support planning and offline analysis. Limitations include model bias, compounding prediction errors, unrealistic simulated states, and distribution shift. If the learned model is inaccurate, the policy may optimize for behaviors that work only inside the model. Mixing real and synthetic data, using short rollouts, uncertainty estimates, and periodic model validation can reduce these risks.

Model-Based Data Generation: 20

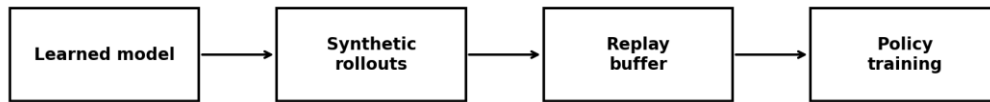


Figure 20.1 — Primary diagram for Question 20.

Component	Role / Process	Expected Outcome
Learned model	Generates predicted transitions	Synthetic experience
Replay buffer	Stores real + model data	Reusable samples
Policy learner	Trains from sampled batches	Better sample efficiency

Table 20.1 — Key components and expected outcomes for the proposed design.

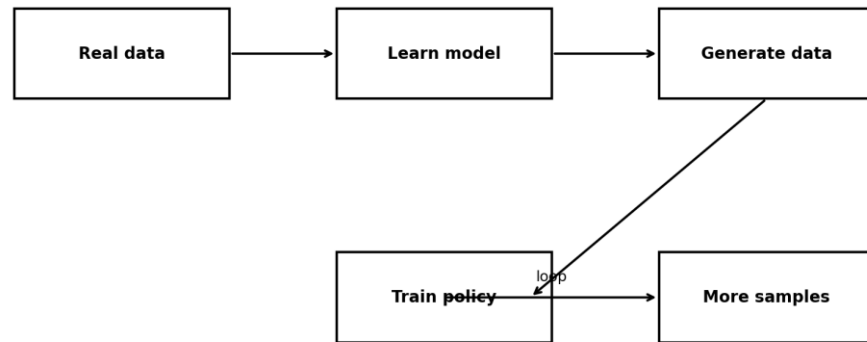
Synthetic Experience Loop

Figure 20.2 — Supporting relationship / process diagram for Question 20.

VALUE-EQUIVALENCE PREDICTION AND POLICY OPTIMIZATION

21. Draw a flowchart explaining value-equivalence prediction for estimating future rewards.

Answer / Design Explanation

Value-equivalence prediction focuses on predicting quantities that matter for decision quality rather than reproducing every detail of the environment. From a state and action, the learned representation predicts future reward or value-related outcomes. The predicted value estimates expected discounted return. The policy evaluator uses these predictions to determine which actions are good. The policy improvement step then favors actions with higher predicted value. By concentrating model learning on decision-relevant information, the approach can reduce unnecessary modeling complexity while supporting policy optimization.

Value-Equivalence / Policy Optimization: 21

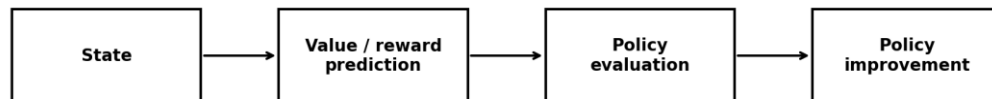


Figure 21.1 — Primary diagram for Question 21.

Component	Role / Process	Expected Outcome
Value prediction	Estimates future return	Decision-relevant signal
Planning / evaluation	Compares candidate behavior	Policy assessment
Policy optimization	Updates toward higher return	Improved policy

Table 21.1 — Key components and expected outcomes for the proposed design.

Unified Model-Based Policy Optimization

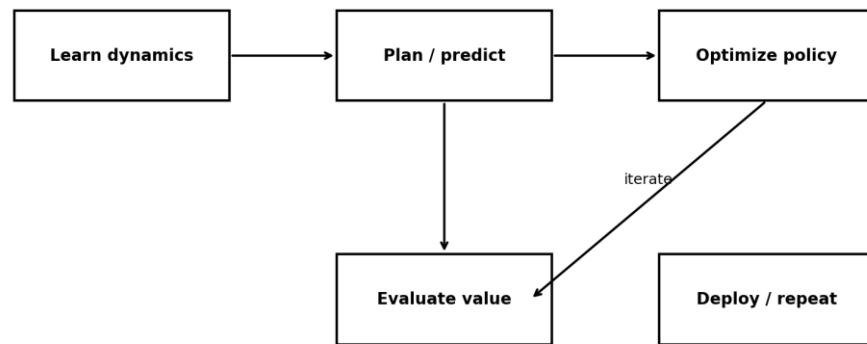


Figure 21.2 — Supporting relationship / process diagram for Question 21.

22. Prepare a concept map showing the relationship among value functions, reward prediction, policy evaluation, and policy improvement.

Answer / Design Explanation

The value function estimates expected cumulative reward from a state under a policy, while an action-value function estimates return after taking a specific action. Reward prediction provides the immediate learning signal, and policy evaluation estimates how well the current policy performs. Policy improvement uses these estimates to select actions or update policy parameters toward higher expected return. The loop is reward prediction → value estimation → policy evaluation → policy improvement → new policy → new experience.

Value-Equivalence / Policy Optimization: 22

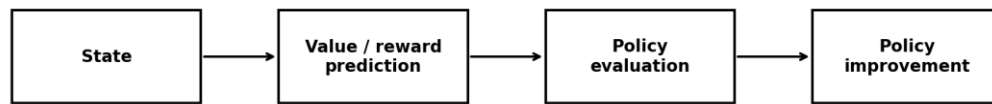


Figure 22.1 — Primary diagram for Question 22.

Component	Role / Process	Expected Outcome
Value prediction	Estimates future return	Decision-relevant signal
Planning / evaluation	Compares candidate behavior	Policy assessment
Policy optimization	Updates toward higher return	Improved policy

Table 22.1 — Key components and expected outcomes for the proposed design.

Unified Model-Based Policy Optimization

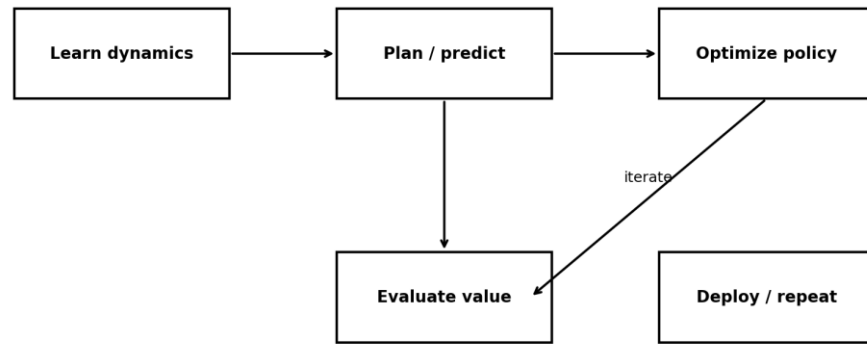


Figure 22.2 — Supporting relationship / process diagram for Question 22.

23. Draw a block diagram illustrating the complete Model-Based Policy Optimization (MBPO) framework.

Answer / Design Explanation

Model-Based Policy Optimization, commonly associated with MBPO, combines a learned dynamics model with a model-free policy learner. Real transitions are collected from the environment and used to train an ensemble or learned dynamics model. Short synthetic rollouts are then generated starting from real states. These model-generated transitions are added to the training data. A policy and value function are updated using the combined experience. The improved policy interacts with the real environment, producing fresh data that refine the model. Short rollouts limit the effect of compounding model errors.

Value-Equivalence / Policy Optimization: 23

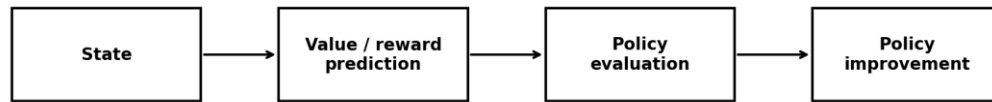


Figure 23.1 — Primary diagram for Question 23.

Component	Role / Process	Expected Outcome
Value prediction	Estimates future return	Decision-relevant signal
Planning / evaluation	Compares candidate behavior	Policy assessment
Policy optimization	Updates toward higher return	Improved policy

Table 23.1 — Key components and expected outcomes for the proposed design.

Unified Model-Based Policy Optimization

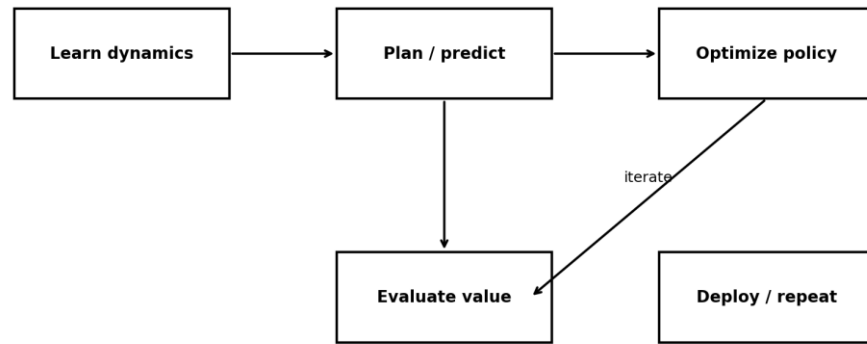


Figure 23.2 — Supporting relationship / process diagram for Question 23.

24. Design a flowchart explaining iterative policy optimization using learned environment dynamics.

Answer / Design Explanation

Iterative policy optimization using learned dynamics begins with real data collection and model learning. The learned dynamics model predicts future states and rewards. The current policy is evaluated using real and simulated experience. A policy optimization algorithm updates parameters to increase expected return. The new policy is tested through additional environment interaction. Differences between real and predicted outcomes are used to refine the model. The cycle continues until performance converges or reaches the required target. Safety constraints and validation should be included before deployment.

Value-Equivalence / Policy Optimization: 24

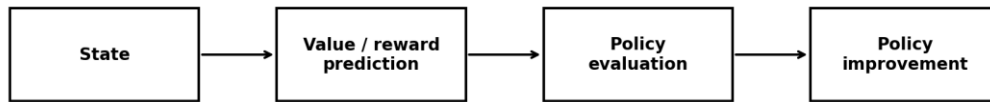


Figure 24.1 — Primary diagram for Question 24.

Component	Role / Process	Expected Outcome
Value prediction	Estimates future return	Decision-relevant signal
Planning / evaluation	Compares candidate behavior	Policy assessment
Policy optimization	Updates toward higher return	Improved policy

Table 24.1 — Key components and expected outcomes for the proposed design.

Unified Model-Based Policy Optimization

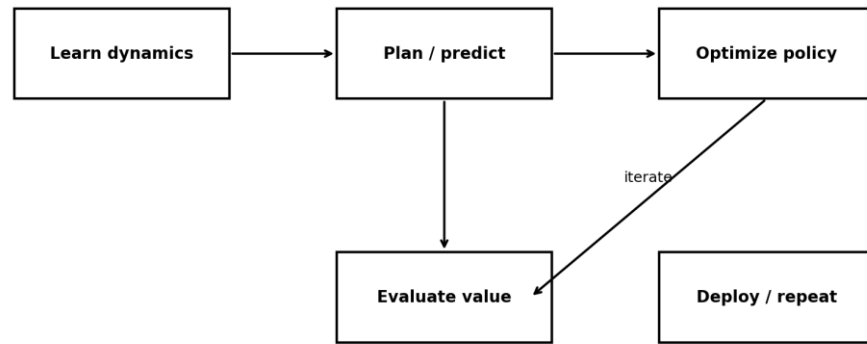


Figure 24.2 — Supporting relationship / process diagram for Question 24.

25. Draw a comprehensive concept map integrating Model-Based RL, Analytic Gradient Computation, Sampling-Based Planning, Model-Based Data Generation, Value-Equivalence Prediction, and Model-Based Policy Optimization into a unified Reinforcement Learning framework.

Answer / Design Explanation

A unified Model-Based RL framework starts with real environment interaction and learning of approximate dynamics. Analytic gradient computation can optimize differentiable policy parameters using reward or value signals. Sampling-based planning can search through model-generated trajectories to identify high-value actions. Model-based data generation can create additional synthetic transitions and improve sample efficiency. Value-equivalence prediction focuses model capacity on quantities relevant to future returns and decisions. Model-Based Policy Optimization combines these ideas by learning dynamics, generating short model rollouts, evaluating value, and updating the policy. The improved policy returns to the environment, producing new data. Together, these components form a loop: Environment → Model Learning → Prediction/Value → Sampling and Planning → Policy Optimization → Execution → New Experience.

Value-Equivalence / Policy Optimization: 25

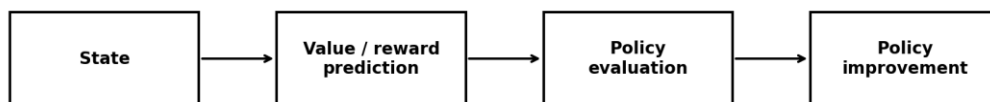
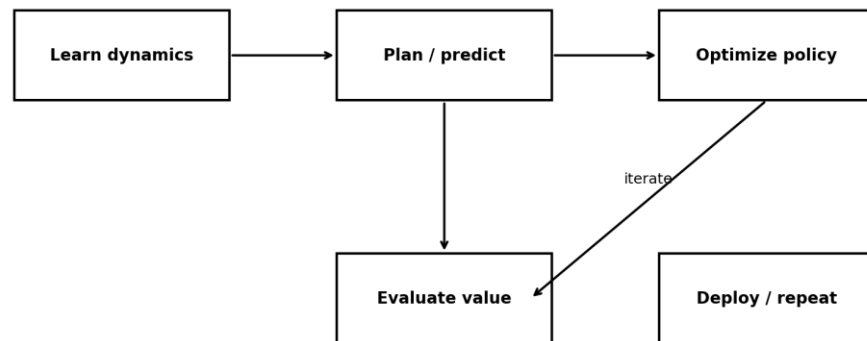


Figure 25.1 — Primary diagram for Question 25.

Component	Role / Process	Expected Outcome
Value prediction	Estimates future return	Decision-relevant signal
Planning / evaluation	Compares candidate behavior	Policy assessment
Policy optimization	Updates toward higher return	Improved policy

*Table 25.1 — Key components and expected outcomes for the proposed design.***Unified Model-Based Policy Optimization***Figure 25.2 — Supporting relationship / process diagram for Question 25.*

